

Newton School of Technology

9

Version Control System

by Raghav Khandelwal

C09: System & AI Essentials

Join the lecture through the online portal

Newton School of Technology

Activity

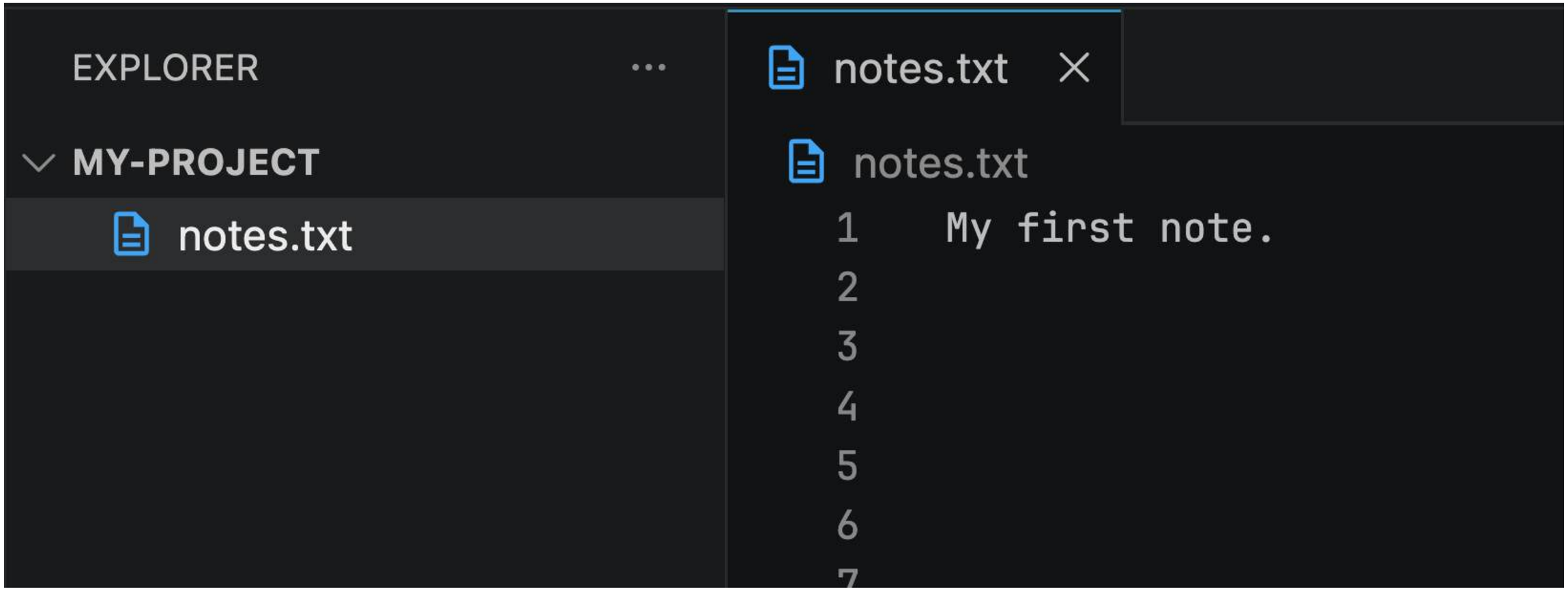
Set Up the Project

Newton School of Technology

```bash
mkdir my-project
cd my-project
touch notes.txt
code .
```

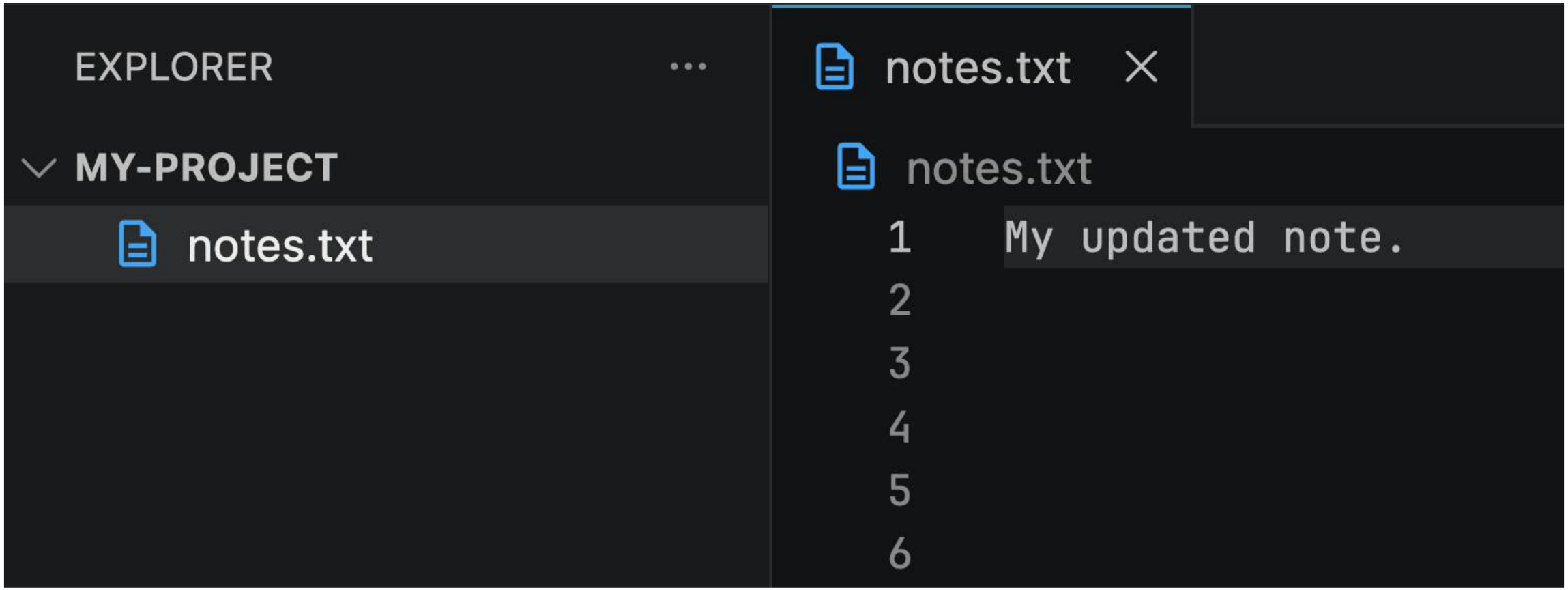
Write the First Note

- Open **notes.txt**. Write the first version of the note.



Update the Note

- Update **notes.txt** and save it.

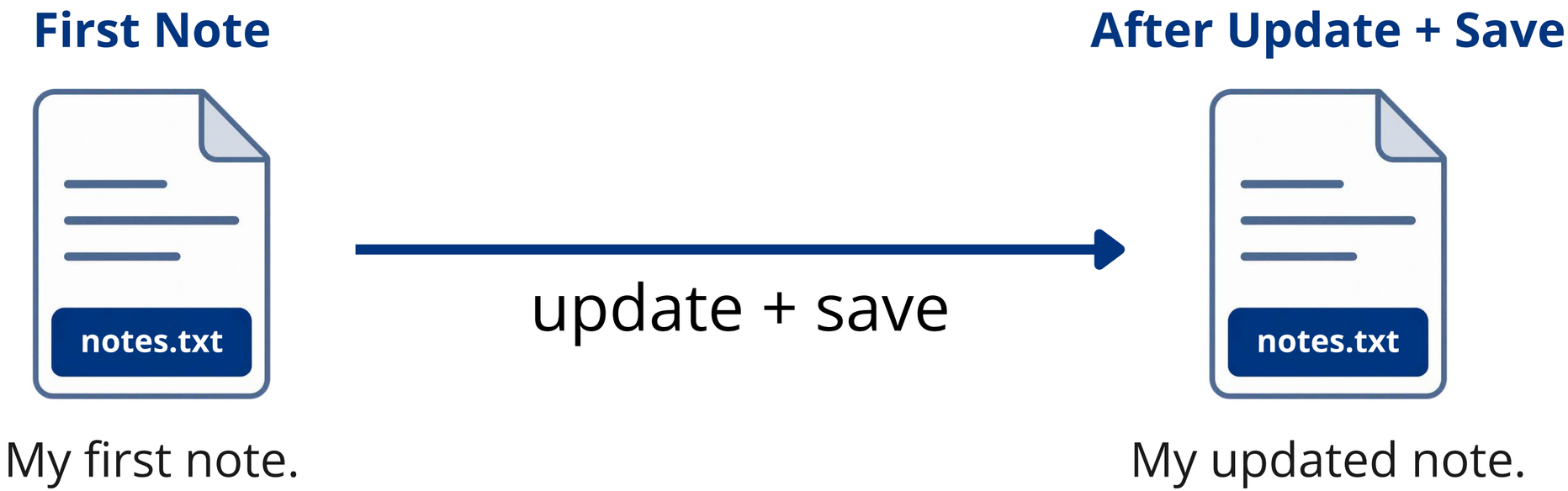


When You Need an Earlier Version

- You update your lab notes, save the file, and later realise that the earlier version had the correct answer.

After Updating and Saving

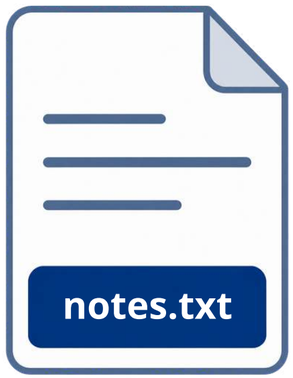
- You update your lab notes, save the file, and later realise that the earlier version had the correct answer.



Only the Latest Version Is Saved

- You update your lab notes, save the file, and later realise that the earlier version had the correct answer.

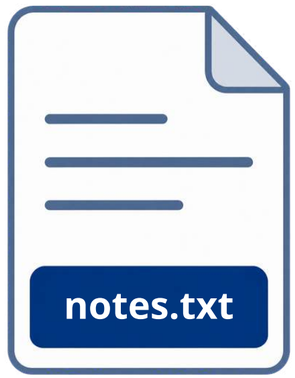
First Note



My first note.

update + save

After Update + Save



My updated note.

Saving keeps the latest version.

The earlier version is not automatically preserved.

Can We Rely on Undo?

Newton School of Technology



Can We Rely on Undo?

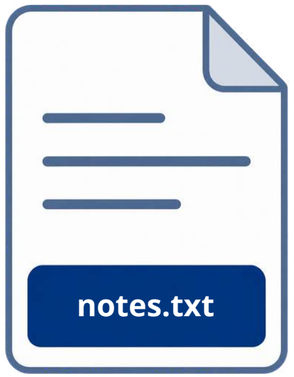
Undo may recover a recent edit, but it is not a reliable project history.

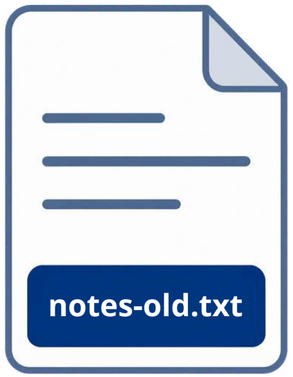
- It depends on the editor and the current session.
- It does not explain what changed or why.
- It does not create named versions you can revisit later.

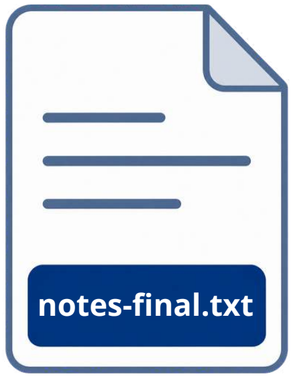


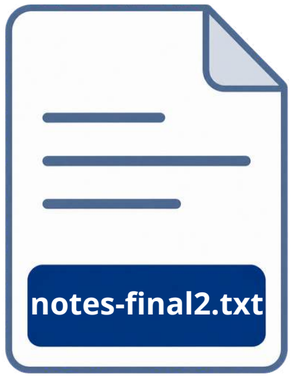
What If We Save Another Copy

Newton School of Technology



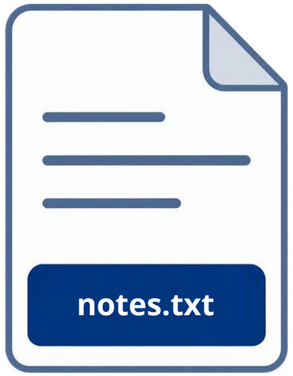


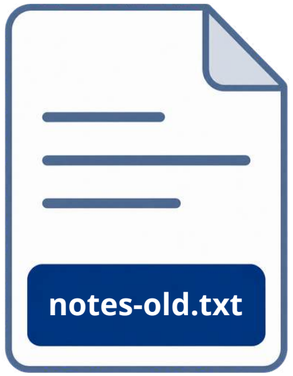


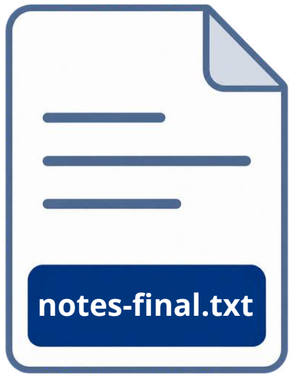


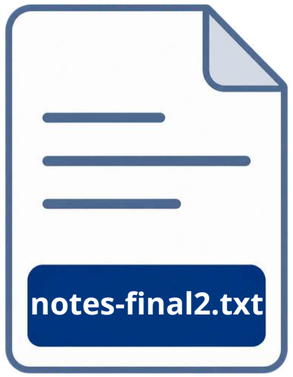
What If We Save Another Copy

- Which file contains the latest version?

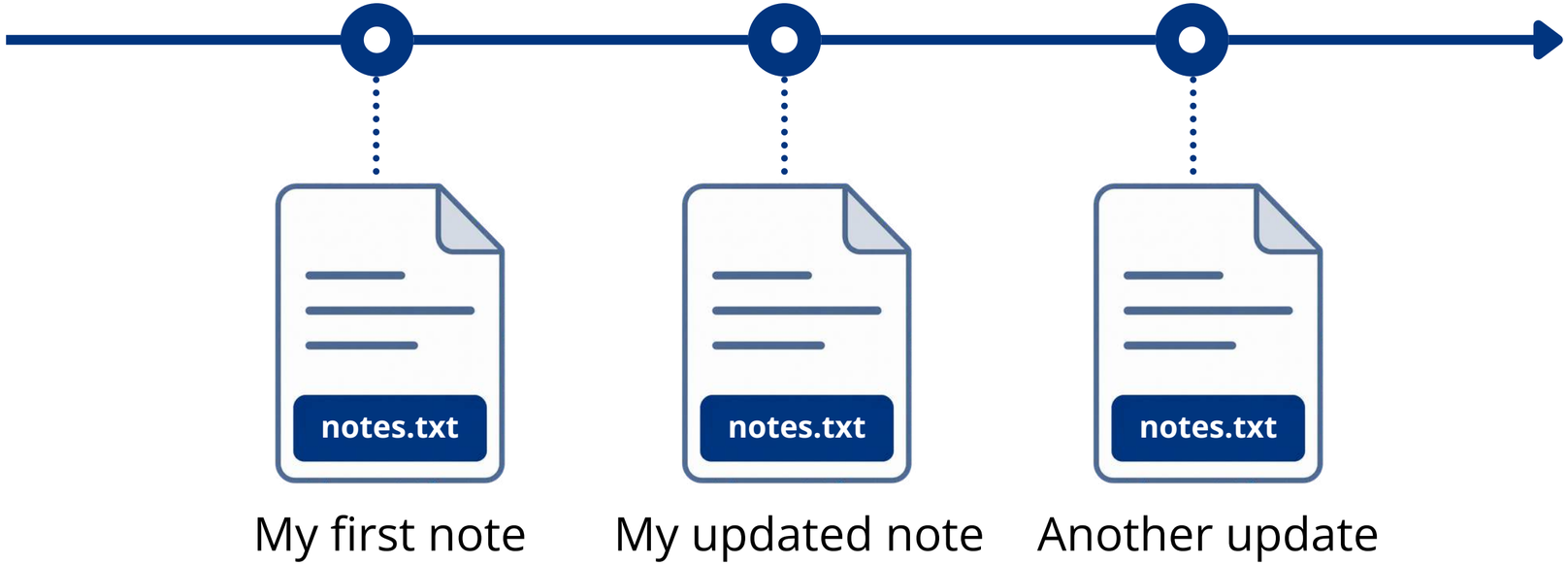




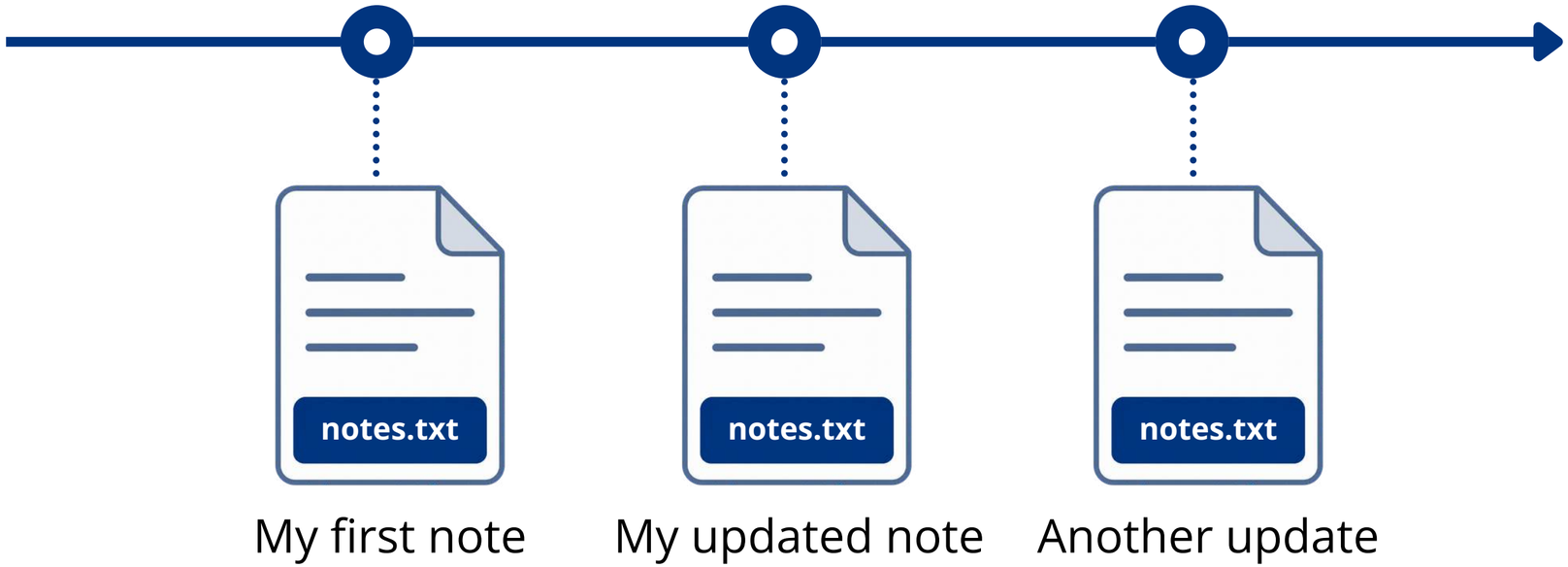




What Makes Project History Useful?



What Makes Project History Useful?



What did it look like before?

What changed?

Why was it changed?

Who changed it?

Newton School of Technology

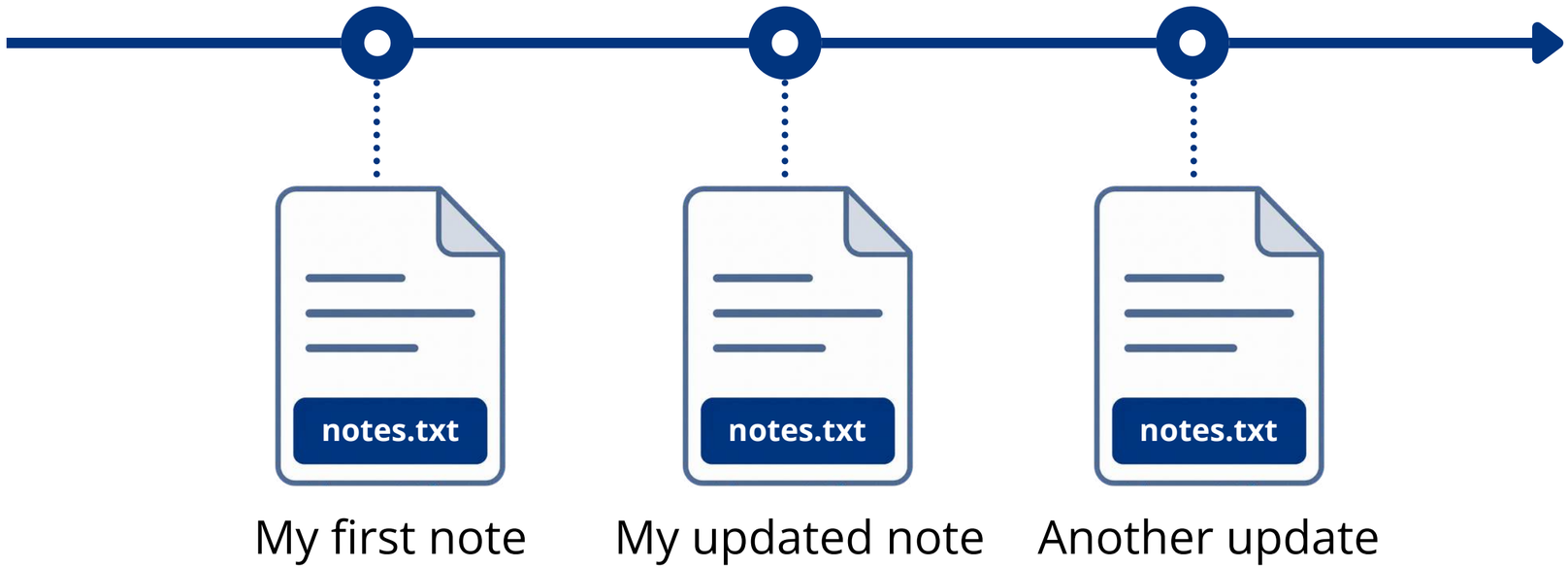
How Do We Preserve Project History?

Version Control

- Version control preserves meaningful project versions as snapshots.
  - A snapshot shows what the project looked like at a particular moment.
- Project history helps us understand what changed and who changed it.

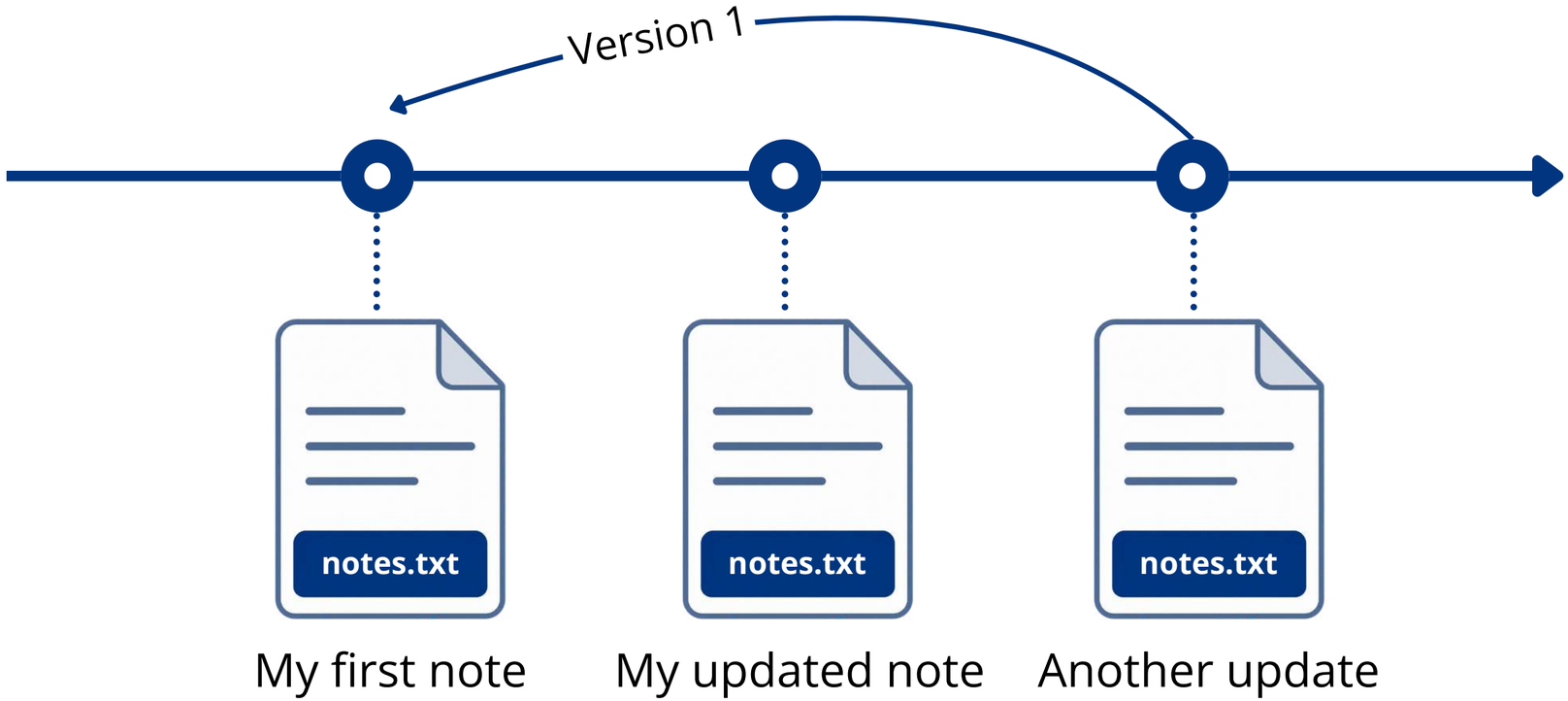
Version Control

- Version control preserves meaningful project versions as snapshots.
  - A snapshot shows what the project looked like at a particular moment.
- Project history helps us understand what changed and who changed it.



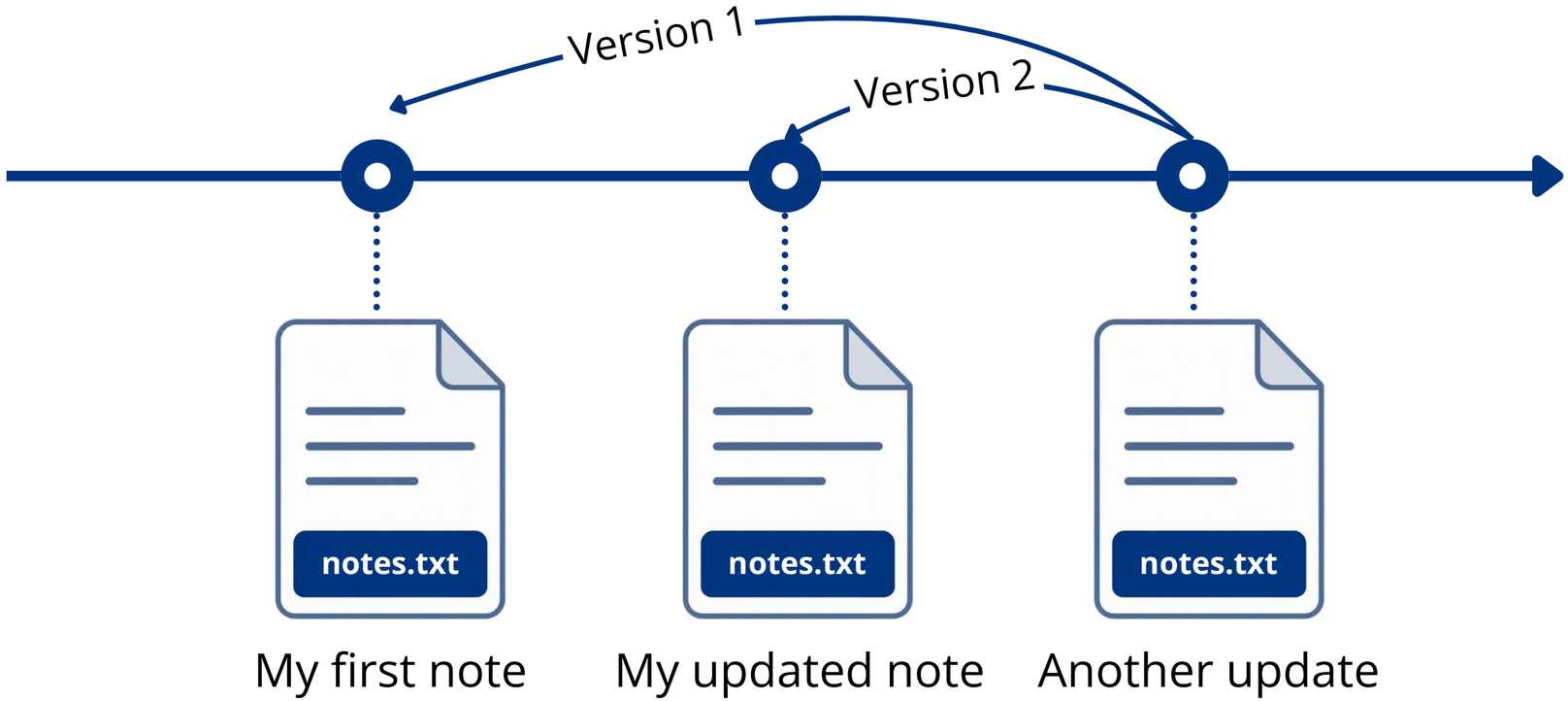
Version Control

- Version control preserves meaningful project versions as snapshots.
  - A snapshot shows what the project looked like at a particular moment.
- Project history helps us understand what changed and who changed it.



Version Control

- Version control preserves meaningful project versions as snapshots.
  - A snapshot shows what the project looked like at a particular moment.
- Project history helps us understand what changed and who changed it.



Version Control Systems

- Different version control systems are available.

Examples





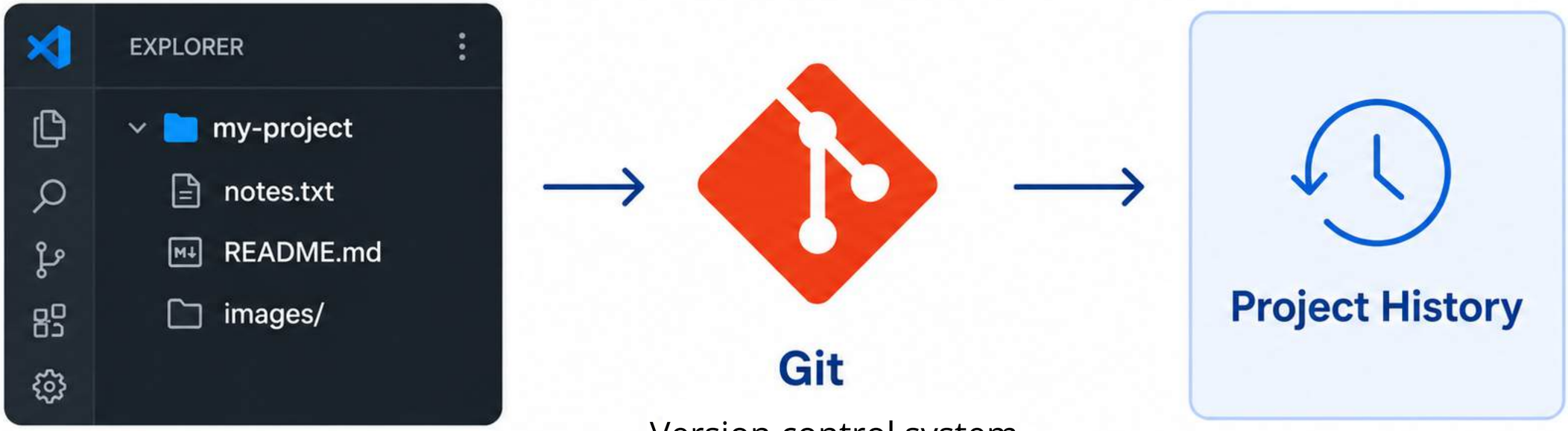


Newton School of Technology

Git is the version control system we will use in this course.

Git

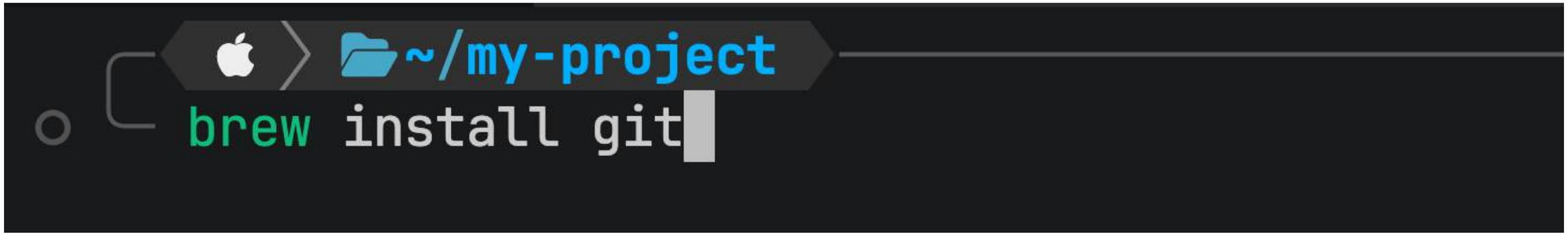
- **Git** is a **version control** system that runs on your computer.
- It tracks and manages changes in project files over time.



Install Git on macOS

Newton School of Technology

1. Install Git using Homebrew



Install Git on macOS

Newton School of Technology

2. Verify the installation



Set Your Git Identity

- Git uses your name and email to identify who made a change.



Check Your Git Identity

Newton School of Technology

- Git records your name and email so the project history shows who made each change.



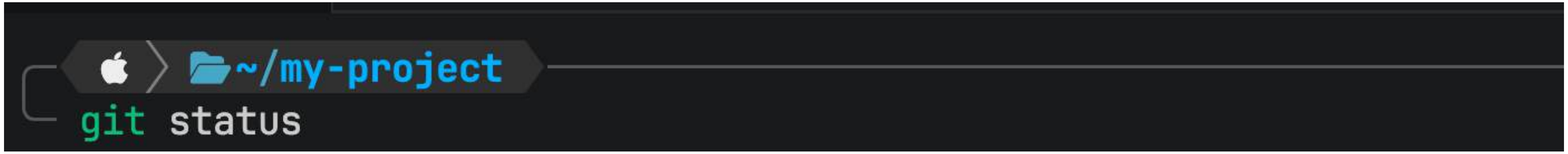
Is Version Control Set Up?

Newton School of Technology

Is Version Control Set Up

Newton School of Technology

- Git is installed, but my-project is still an ordinary folder.



Newton School of Technology

How Do We Set Up Git?

Set Up Git in the Project

Newton School of Technology

```
~/my-project
git init
Initialized empty Git repository in /Users/newtonuser/my-project/.git/
```

What Did git init Create?

- A hidden folder called **.git** has appeared.



The .git Folder

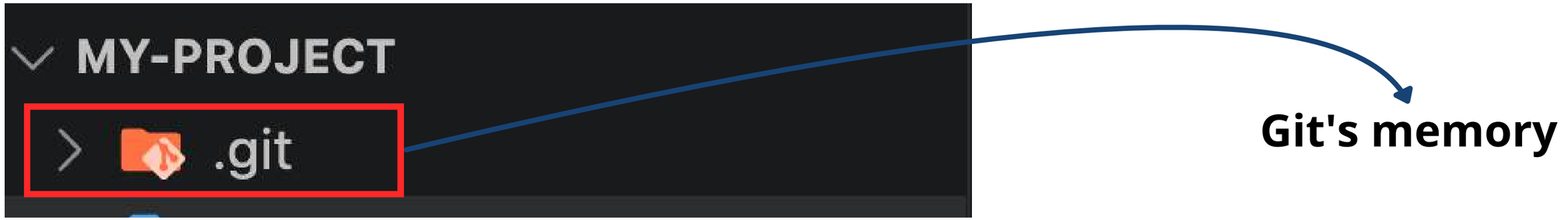
- **git init** creates a hidden **.git** folder inside the project.
- **.git** stores the data Git needs to manage the project history.



Git's memory

The .git Folder

- **git init** creates a hidden **.git** folder inside the project.
- **.git** stores the data Git needs to manage the project history.
- A folder initialized with Git is called a **Git repository.**
- **notes.txt** is still untracked, so Git has not saved it in the project history yet.
    - ⚠️ Do not edit or delete **.git** manually.



Newton School of Technology

What Is the Project’s Current State?

Check the Repository Status

Newton School of Technology



Check the Repository Status

Newton School of Technology

```
~/my-project  git main ?1
git status
On branch main

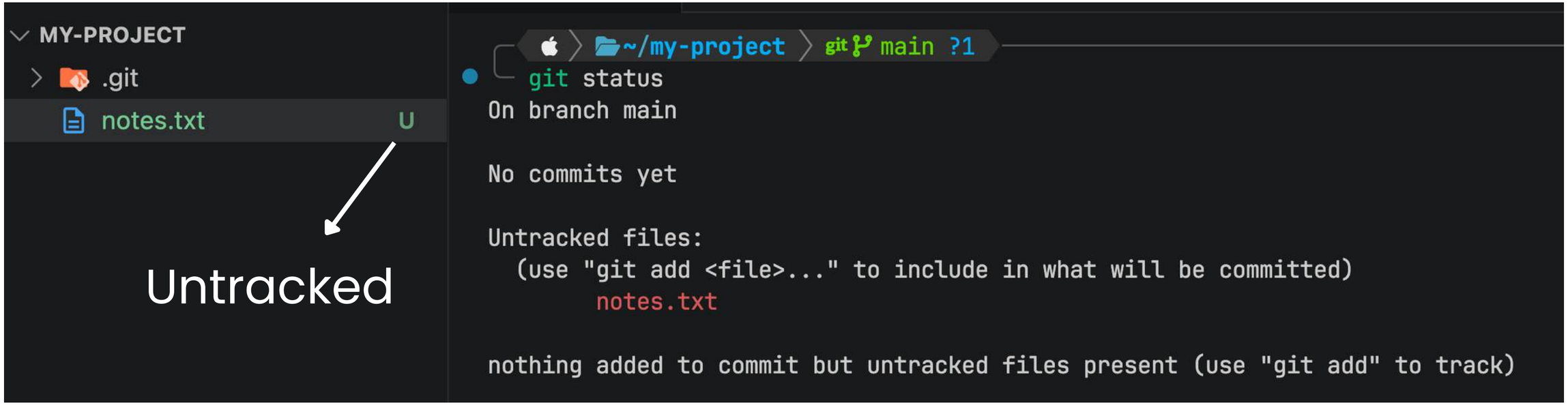
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt

nothing added to commit but untracked files present (use "git add" to track)
```

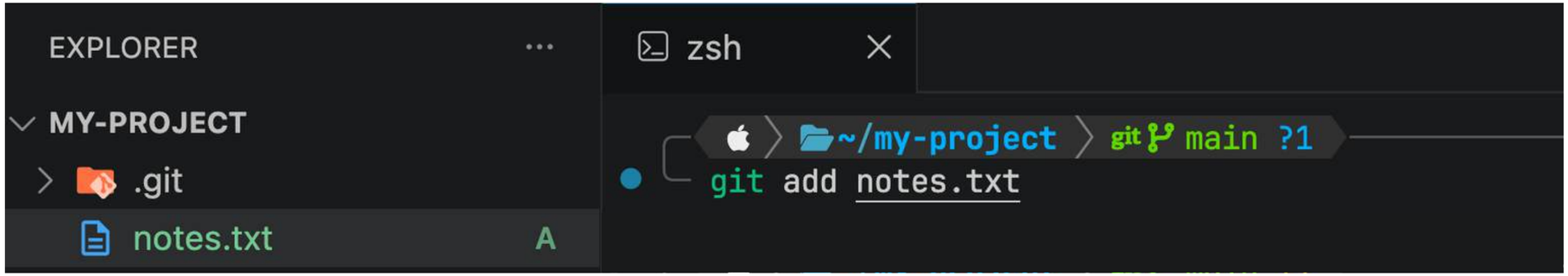
What Does “Untracked” Mean?

- **Untracked** means the file is inside the project folder, but Git has not started following its changes.
- Git can see notes.txt, but it is not included in the project's history yet.



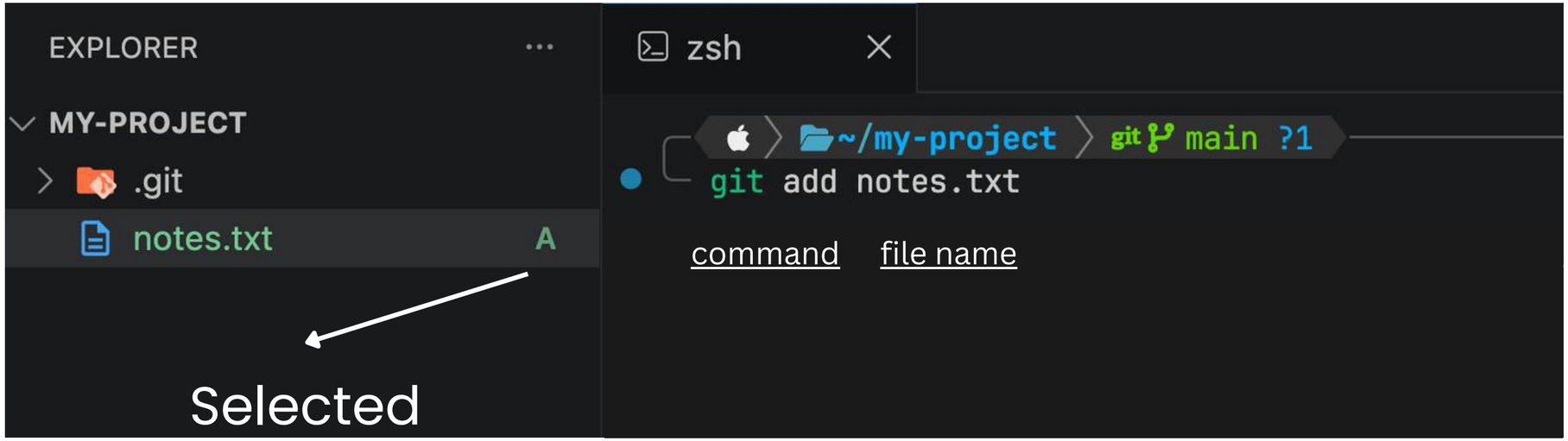
git add

Newton School of Technology



git add

- **git add** selects the current changes in a file.
- Here, we are selecting the current version of **notes.txt.**
- Because this file was untracked, Git will now begin tracking it.



Newton School of Technology

Did Git Select the File?

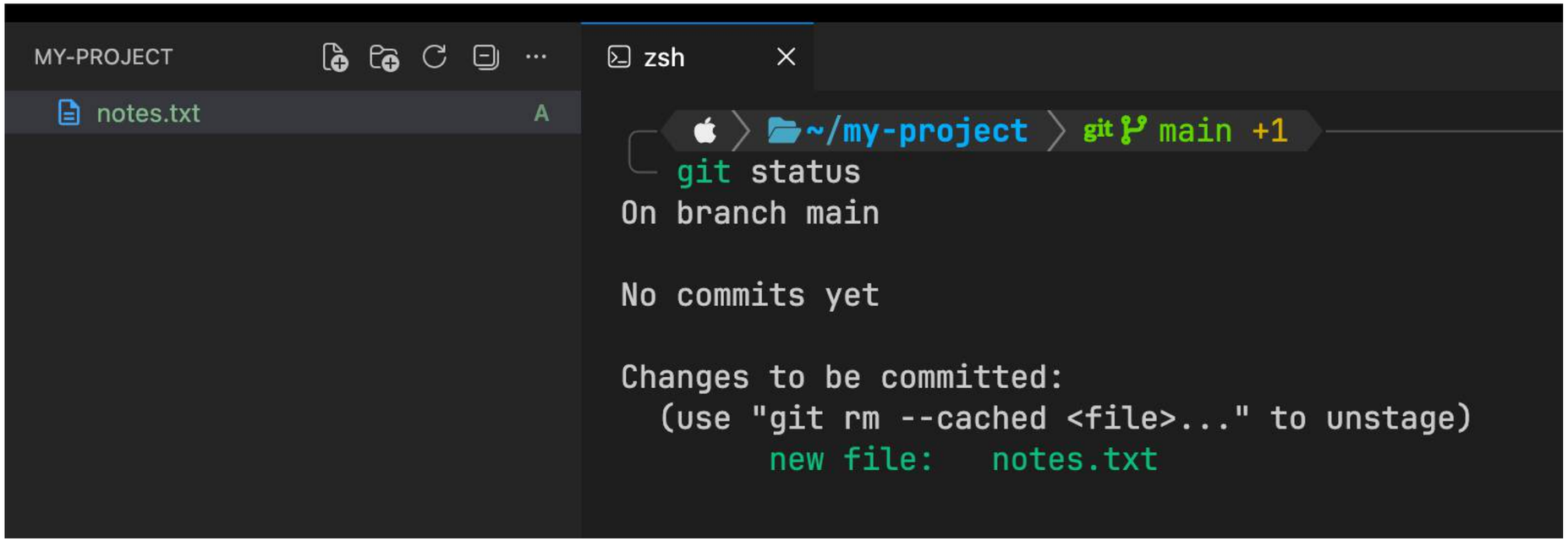
Newton School of Technology

Did Git Select the File?

git add showed no message. Let’s check the file’s status.

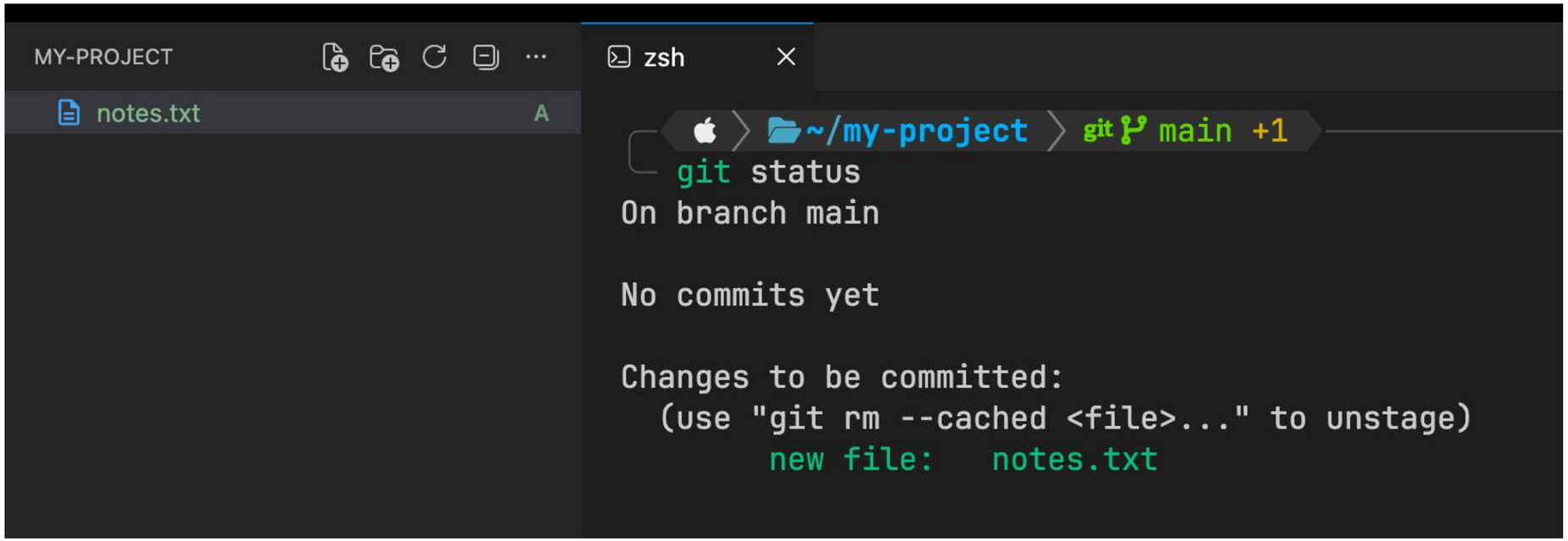
Verify With Status

Newton School of Technology



Verify with status

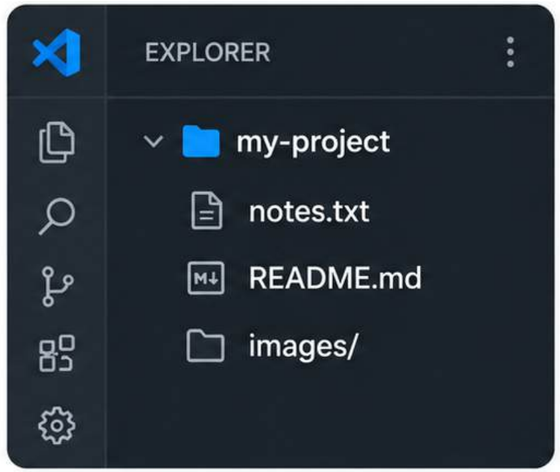
- **notes.txt** now appears under Changes to be committed.
- It is shown in green as a new file.
- Git calls this state **staging**.



Staging

- **git add notes.txt** selects the current version of **notes.txt** for the next snapshot.

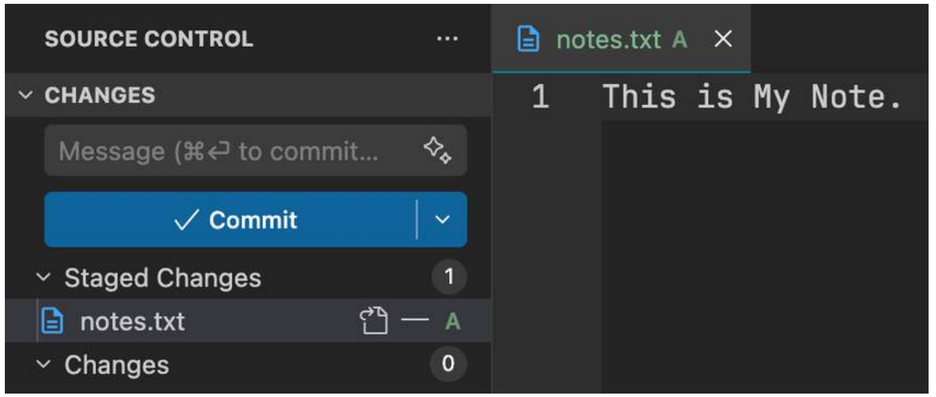
Working Directory



`git add notes.txt`

Staging Area

(Ready for next snapshot)

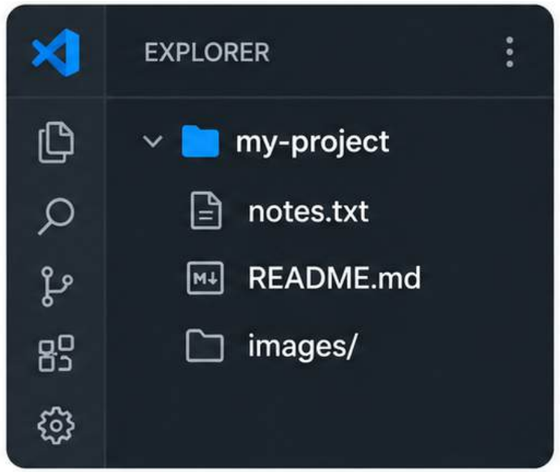


- **Staging area**: Changes selected for the next snapshot.

Staging

- **git add notes.txt** selects the current version of **notes.txt** for the next snapshot.

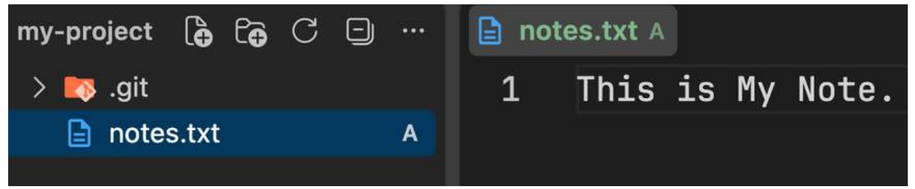
Working Directory



`git add notes.txt`

Staging Area

(Ready for next snapshot)



- **Staging area**: Changes selected for the next snapshot.

Newton School of Technology

The change is staged.
How do we record it in project history?

Commit

- A **commit** is a saved snapshot of the staged changes.
- Git adds this snapshot to the project’s history.
- Each commit has a message that describes the saved changes.

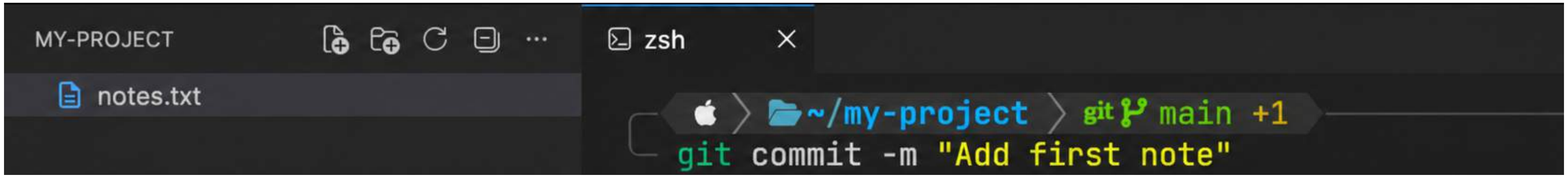
Commit

- A **commit** is a saved snapshot of the staged changes.
- Git adds this snapshot to the project's history.
- Each commit has a message that describes the saved changes.



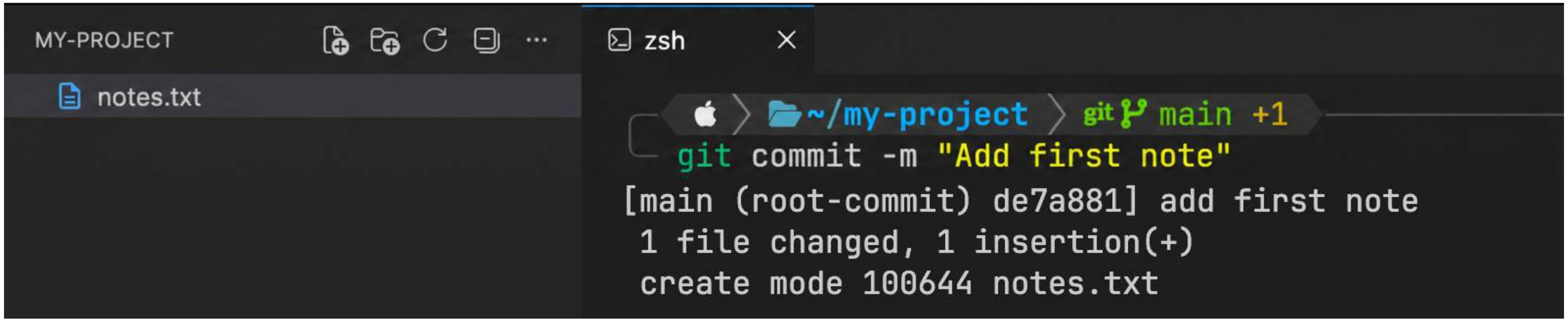
Create a Commit

Newton School of Technology



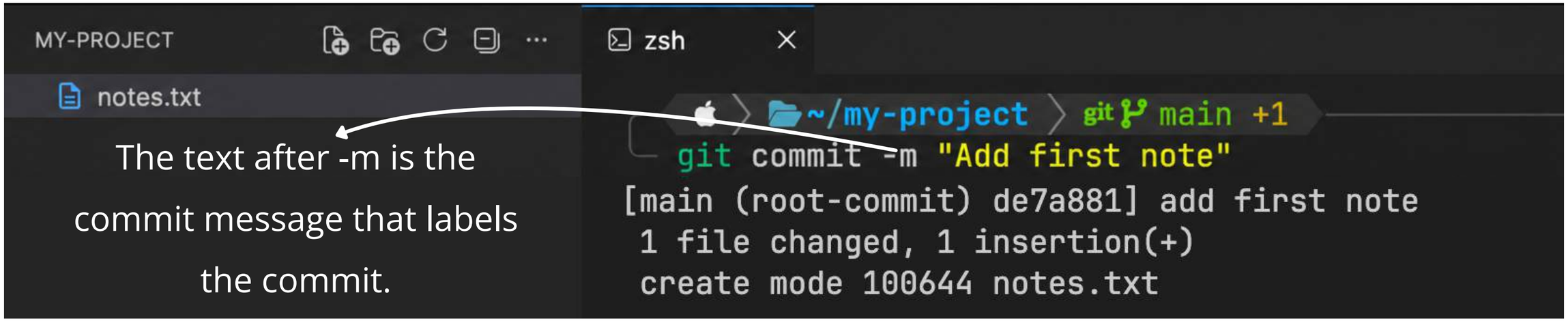
Create a Commit

- git commit records the staged changes.
- -m lets us write a short message.
- "Add first note" describes the saved changes.



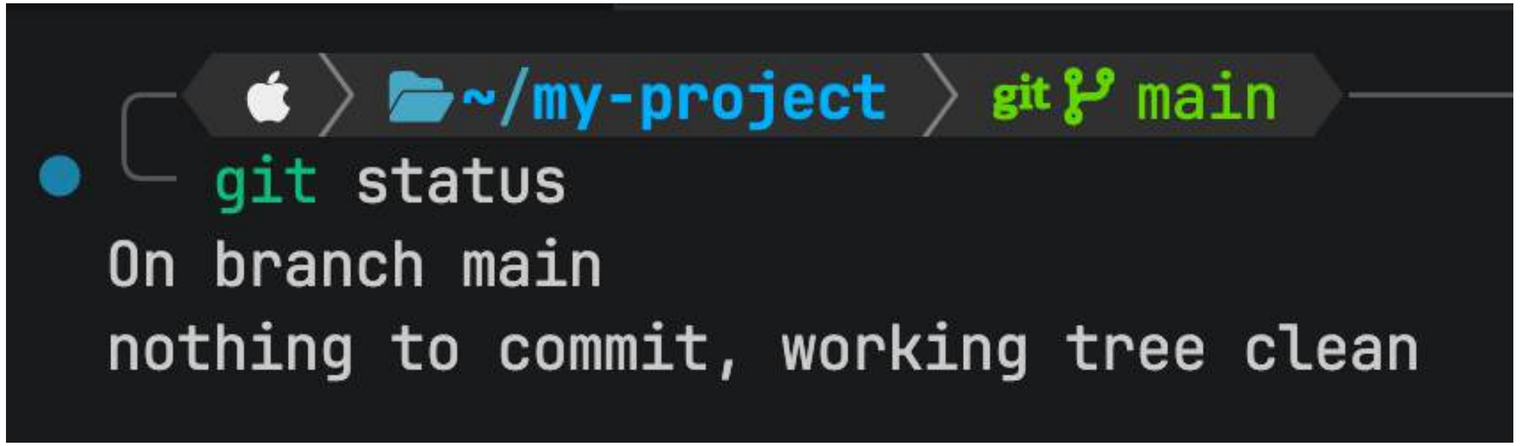
Create a Commit

- git commit records the staged changes.
- -m lets us write a short message.
- "Add first note" describes the saved changes.



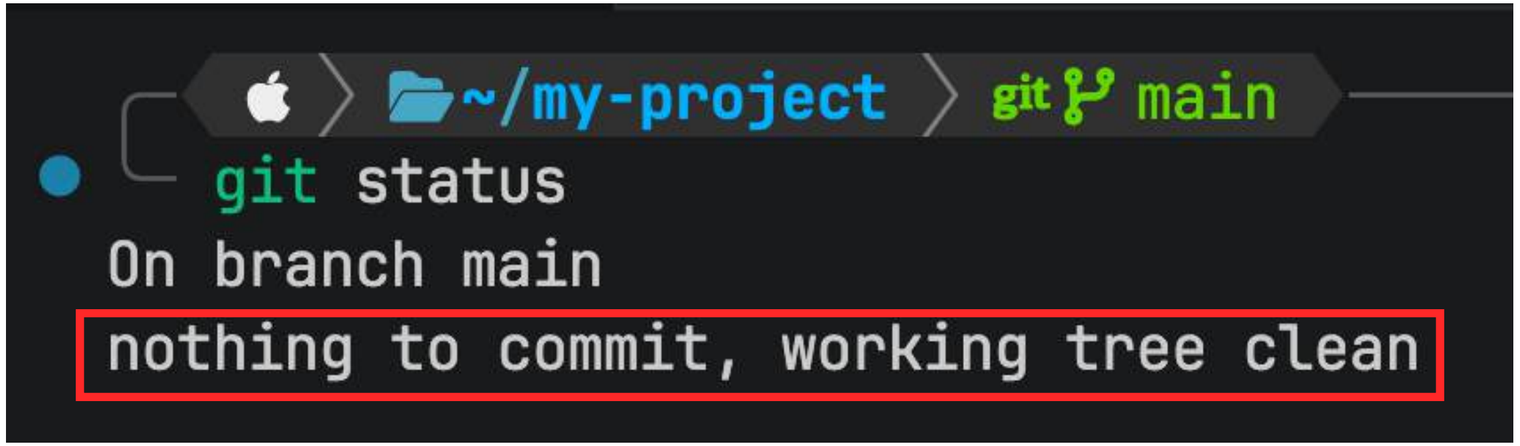
Check the Project Status

Newton School of Technology



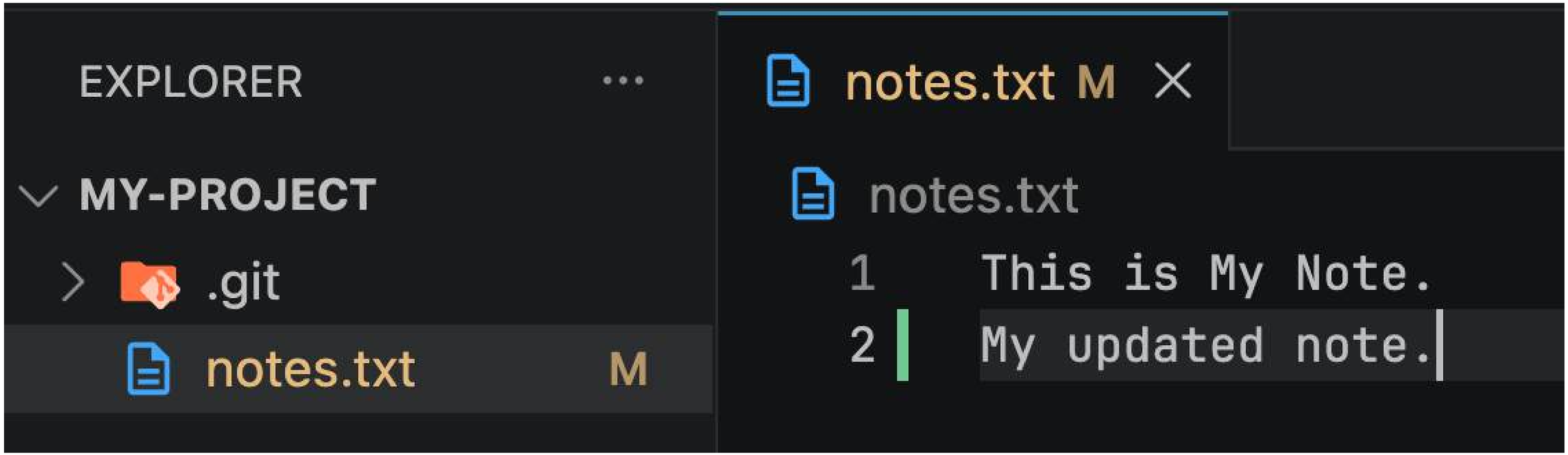
Check the Project Status

- Git checks the project after the commit.
- There are no new changes to save.
- working tree clean means the project matches the latest commit.



Change the Project Again

- Open notes.txt.
- Keep the first line unchanged.
- Add one new line.



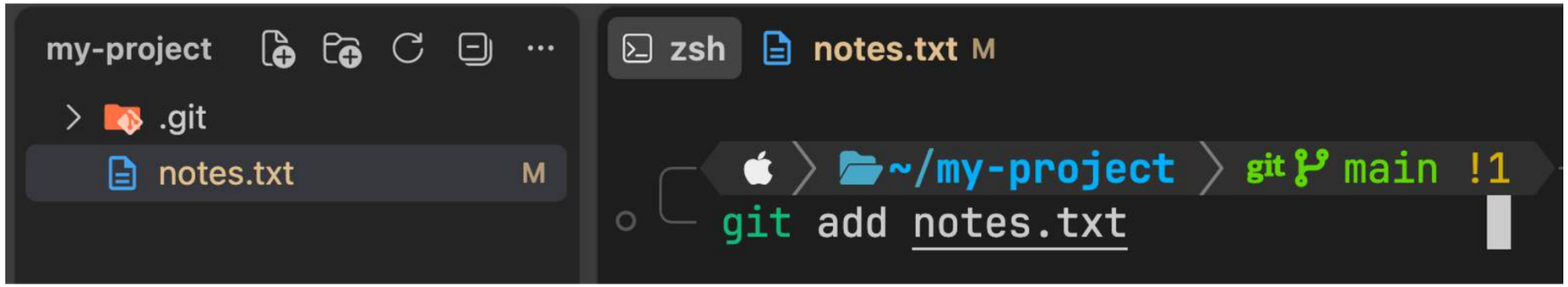
Check the New Change

- Git checks the project again.
- **notes.txt** is marked as **modified**.
- This means the file changed after the **last commit.**

```
~/my-project  git main !1
git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   notes.txt
```

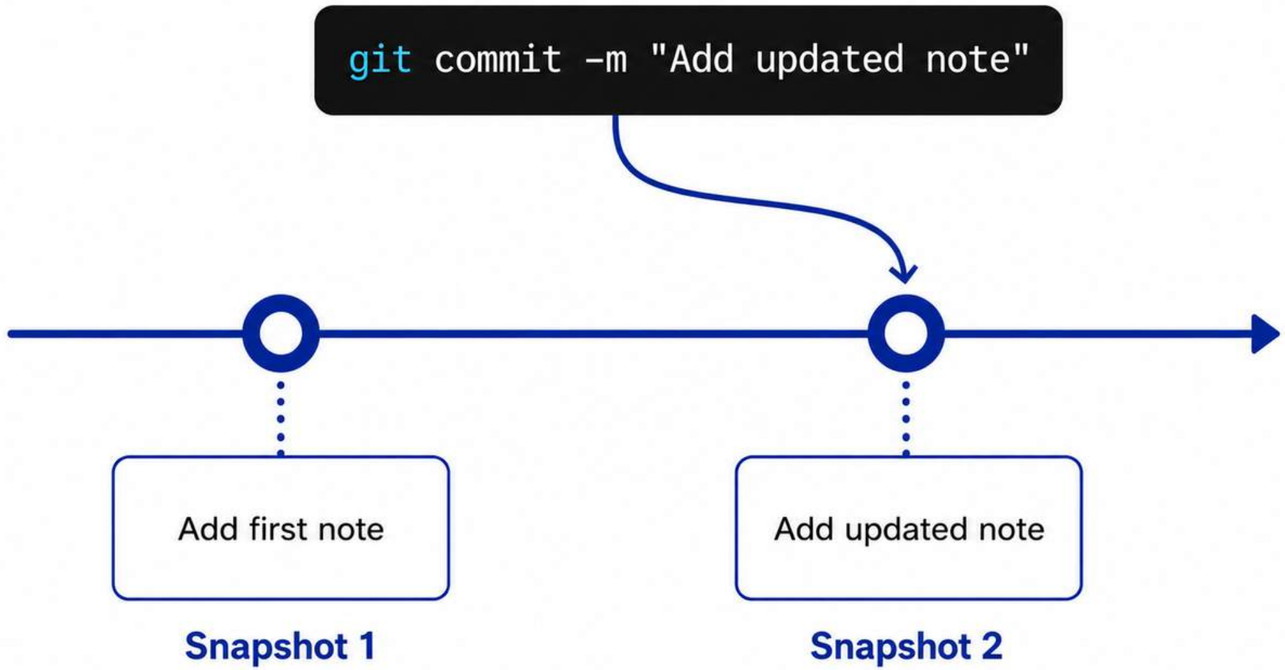
Stage the New Change

- **git add** selects the latest changes in **notes.txt.**
- The new line is now selected for the next commit.



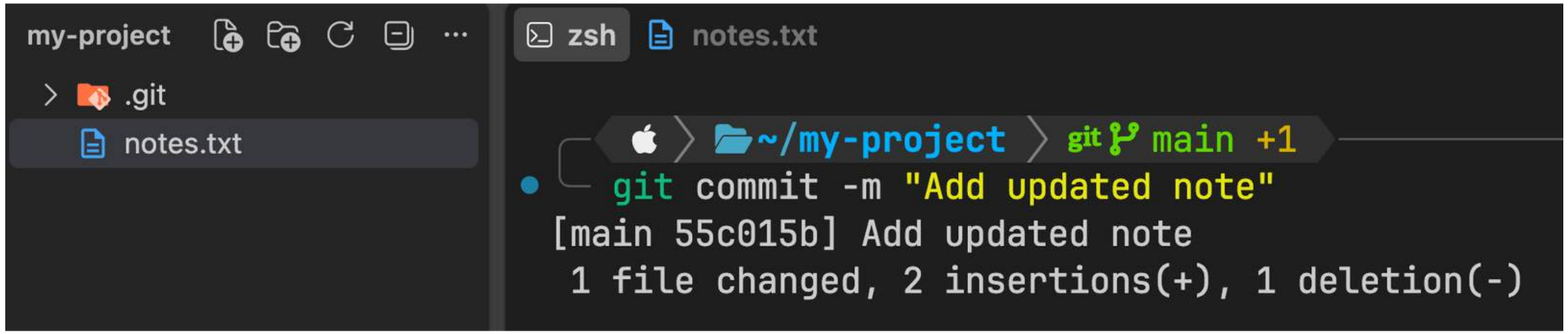
Create the Second Commit

- This commit saves the new line in **notes.txt.**
- The project now has two saved **snapshots**.



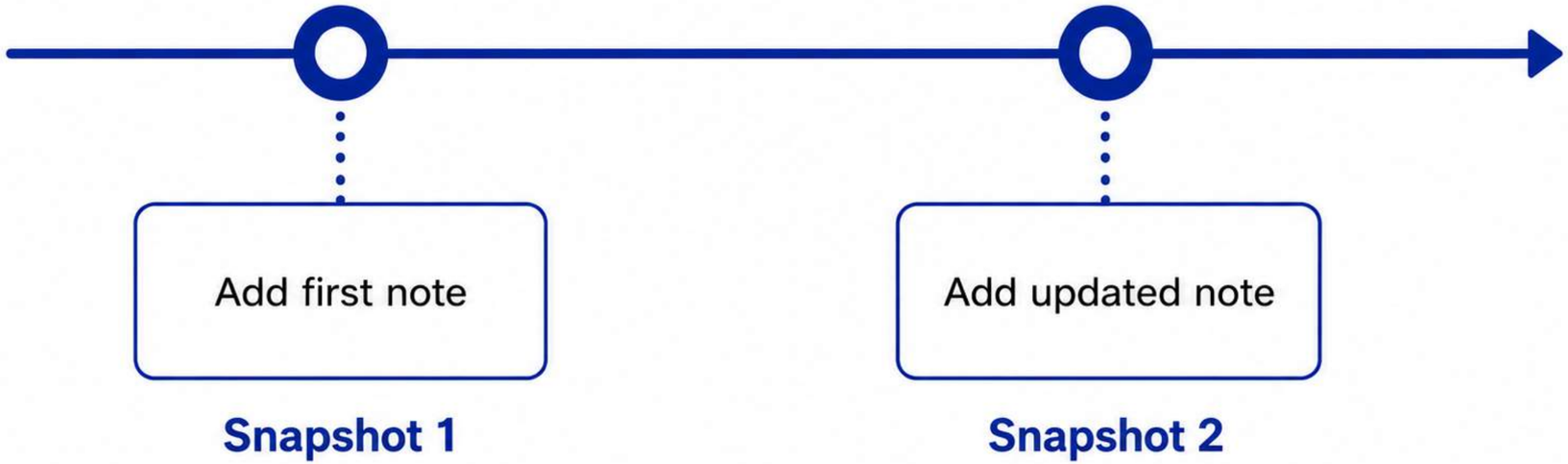
Create the Second Commit

- This commit saves the new line in **notes.txt.**
- The project now has two saved **snapshots**.



The Project Has Two Commits

- The first commit saved the original note.
- The second commit saved the updated note.
- Both saved snapshots are now part of the project history.



Newton School of Technology

How Can We See the Project History?

Git Has a Command for Reading History

Newton School of Technology

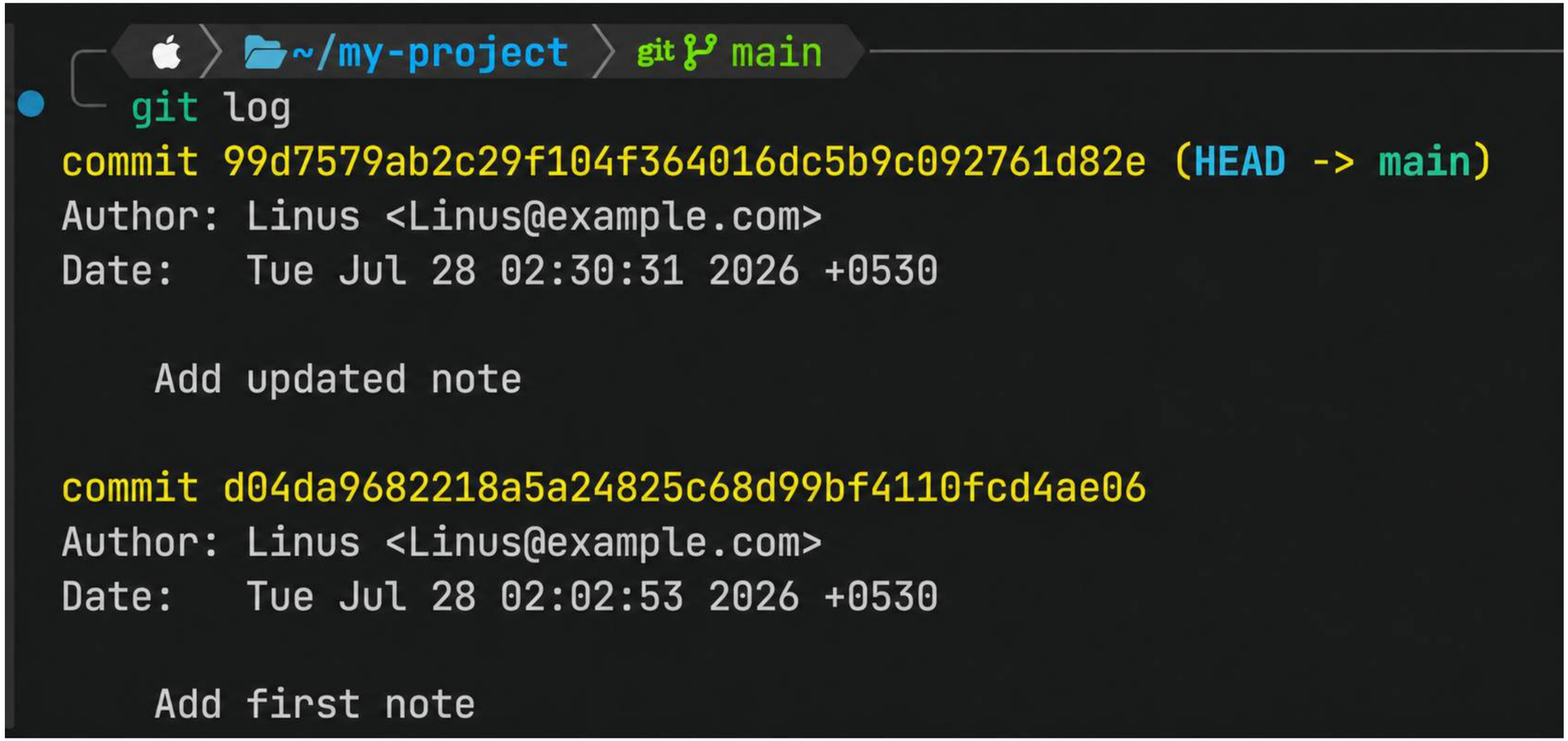
- **git log** shows the commits in the project.
- Each entry includes a commit ID, author, date, and message.



Git Has a Command for Reading History

Newton School of Technology

- **git log** shows the commits in the project.
- Each entry includes a commit ID, author, date, and message.



Newton School of Technology

Now that you know the workflow, what are the steps you performed?

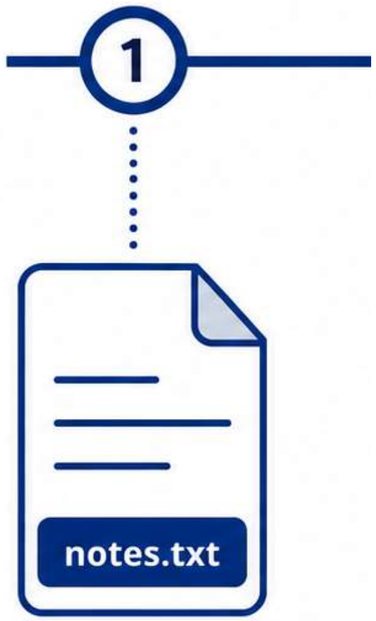
Can You Rebuild the Workflow?

Newton School of Technology

Can You Rebuild the Workflow

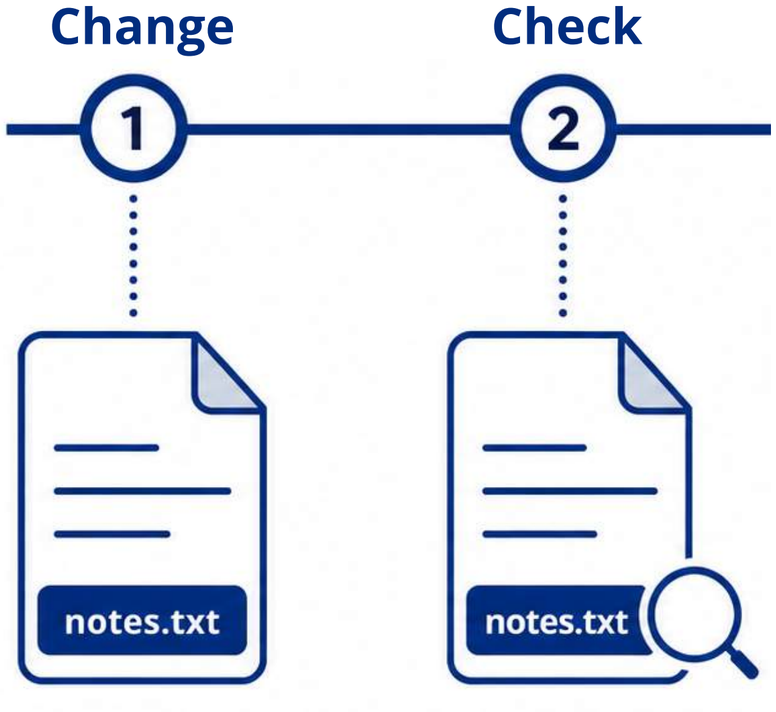
Newton School of Technology

Change



Make a change to the file.

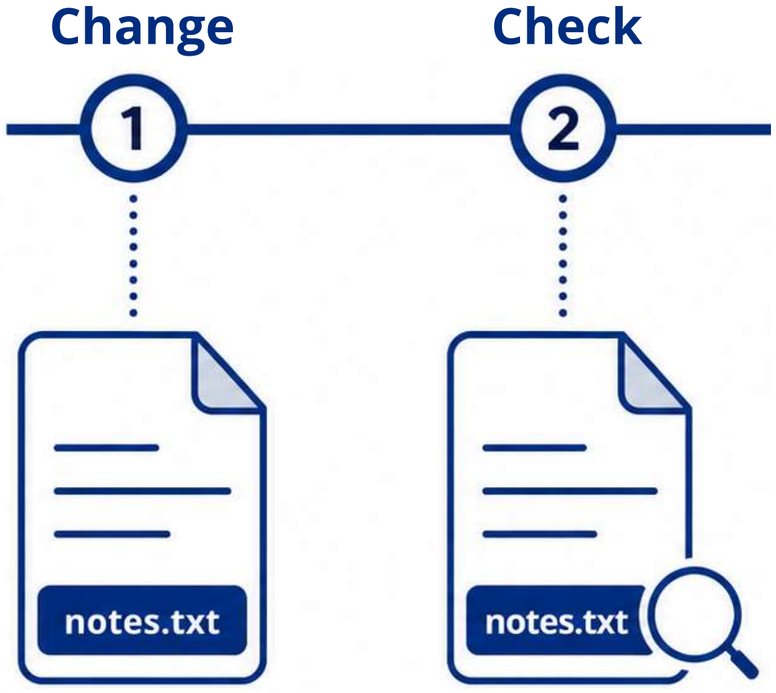
Can You Rebuild the Workflow



Make a change to the file.

Which command checks Git's status?

Can You Rebuild the Workflow

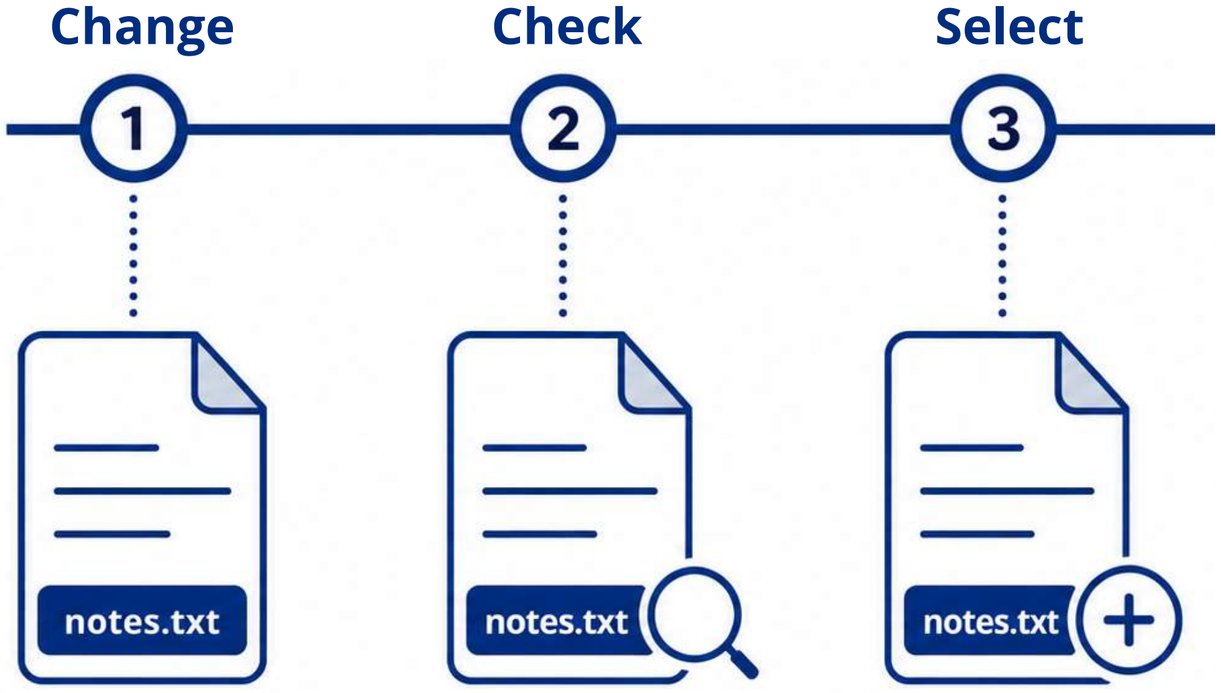


Make a change to the file.

Which command checks Git's status?

git status

Can You Rebuild the Workflow



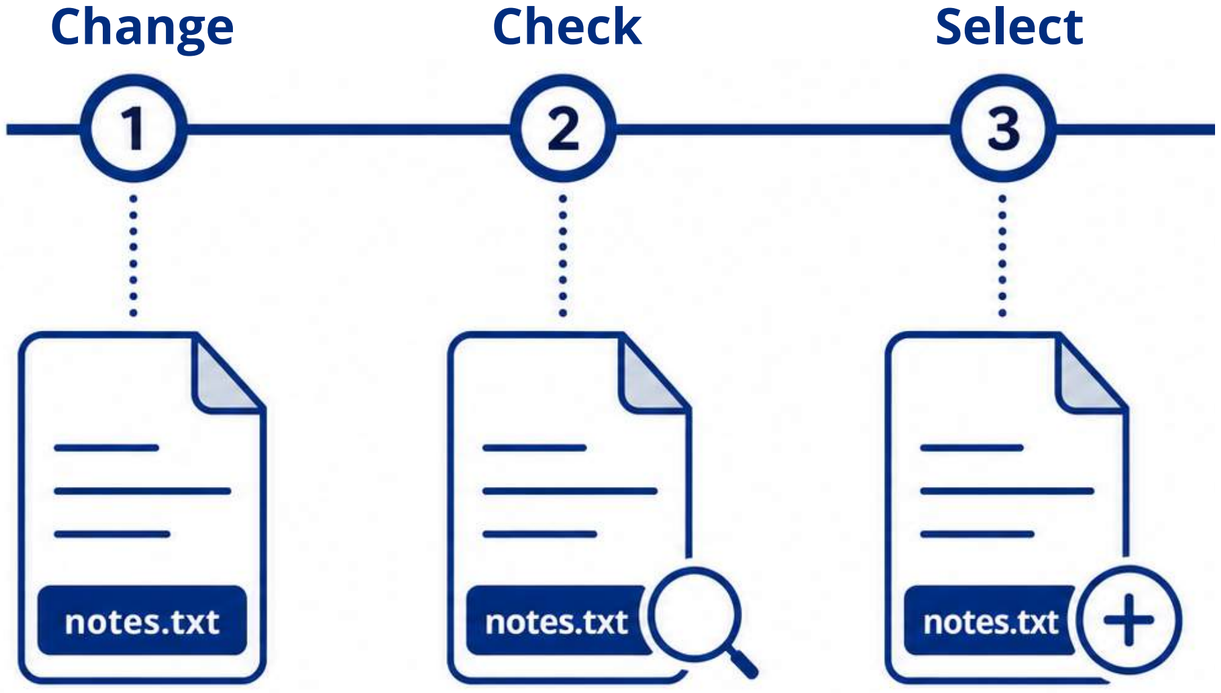
Make a change to the file.

Which command checks Git's status?

git status

How do you select the change?

Can You Rebuild the Workflow



Make a change to the file.

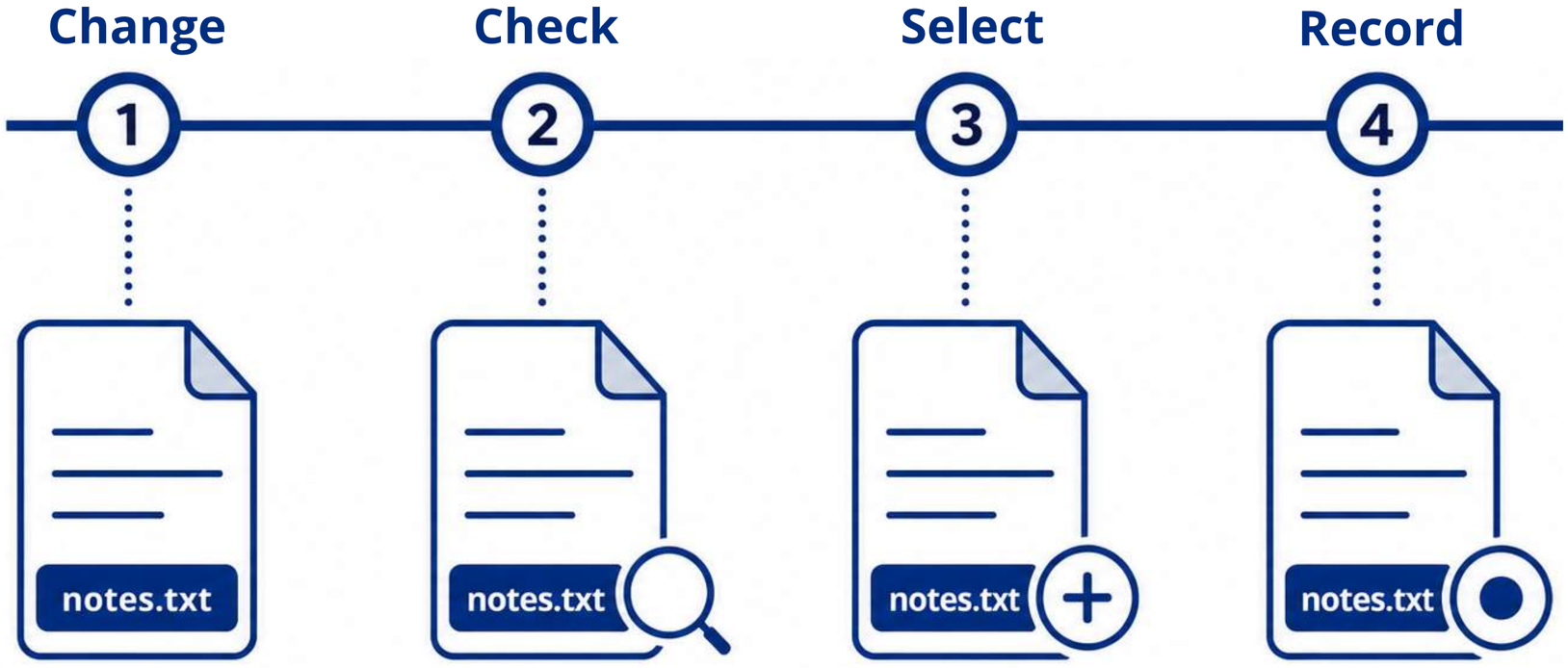
Which command checks Git's status?

git status

How do you select the change?

git add

Can You Rebuild the Workflow



Make a change to the file.

Which command checks Git's status?

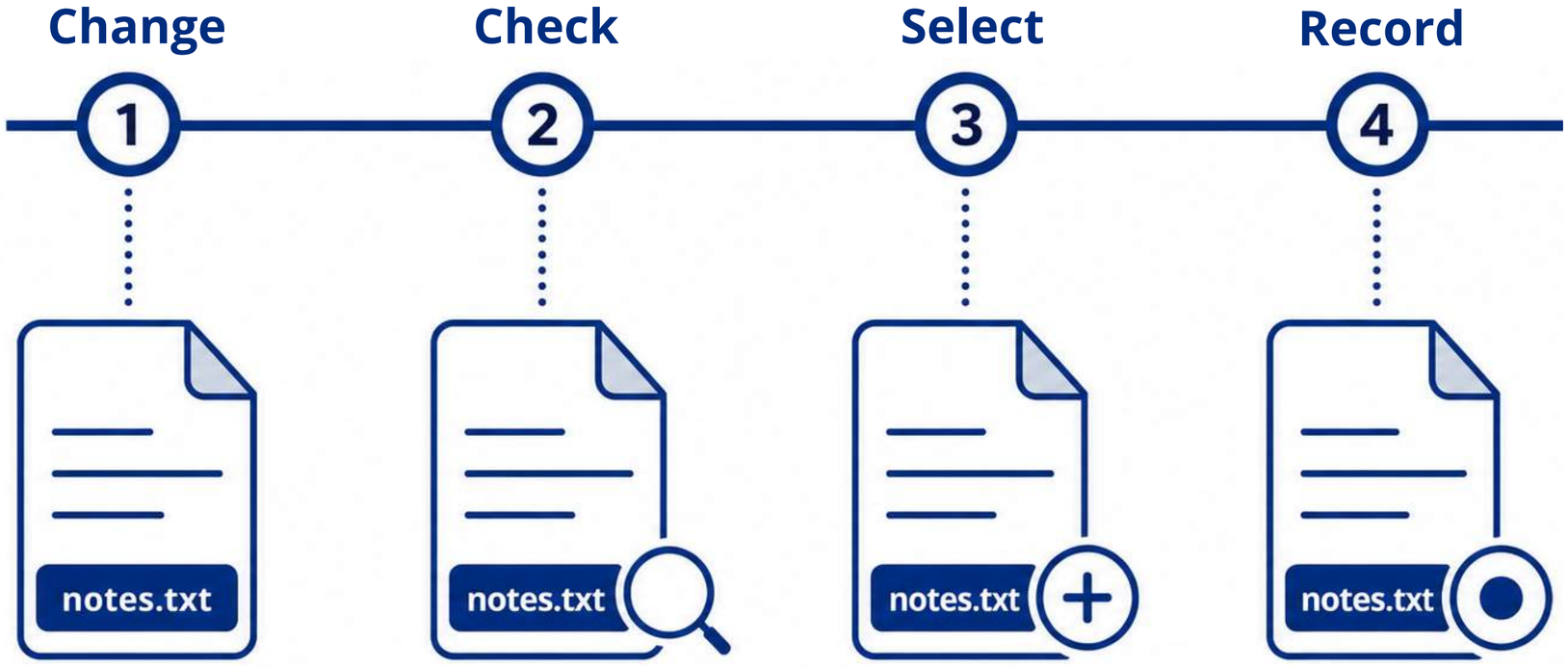
git status

How do you select the change?

git add

How do you record the change?

Can You Rebuild the Workflow



Make a change to the file.

Which command checks Git's status?

git status

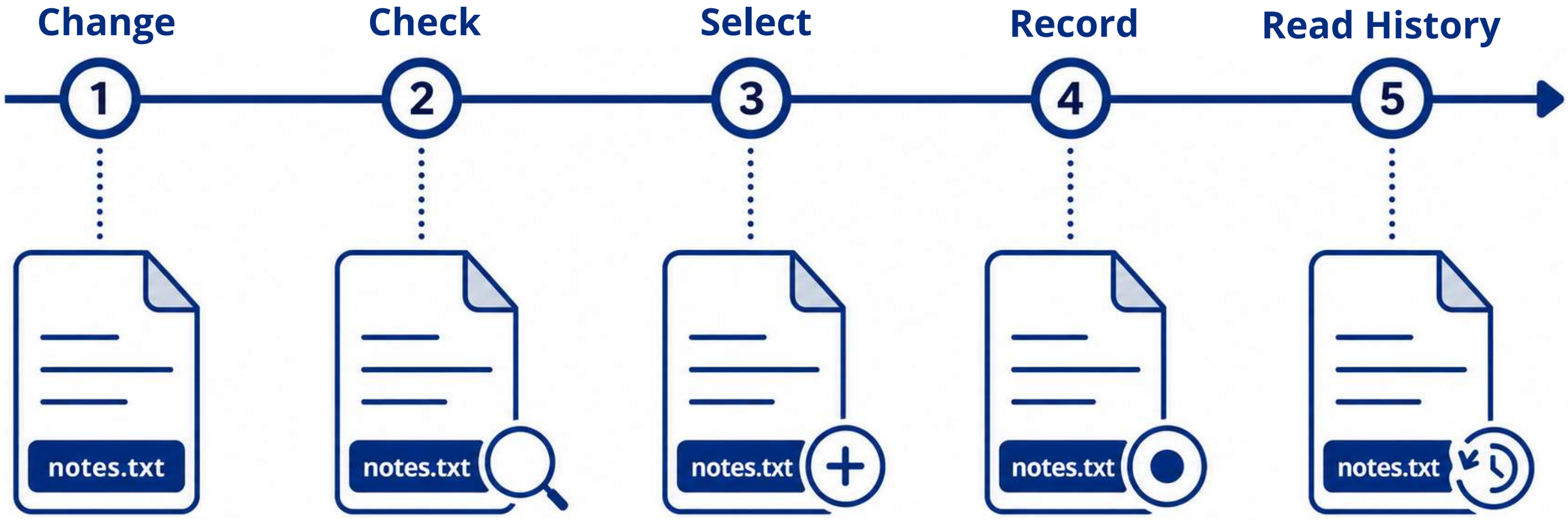
How do you select the change?

git add

How do you record the change?

git commit -m “<msg>”

Can You Rebuild the Workflow



Make a change to the file.

Which command checks Git's status?

git status

How do you select the change?

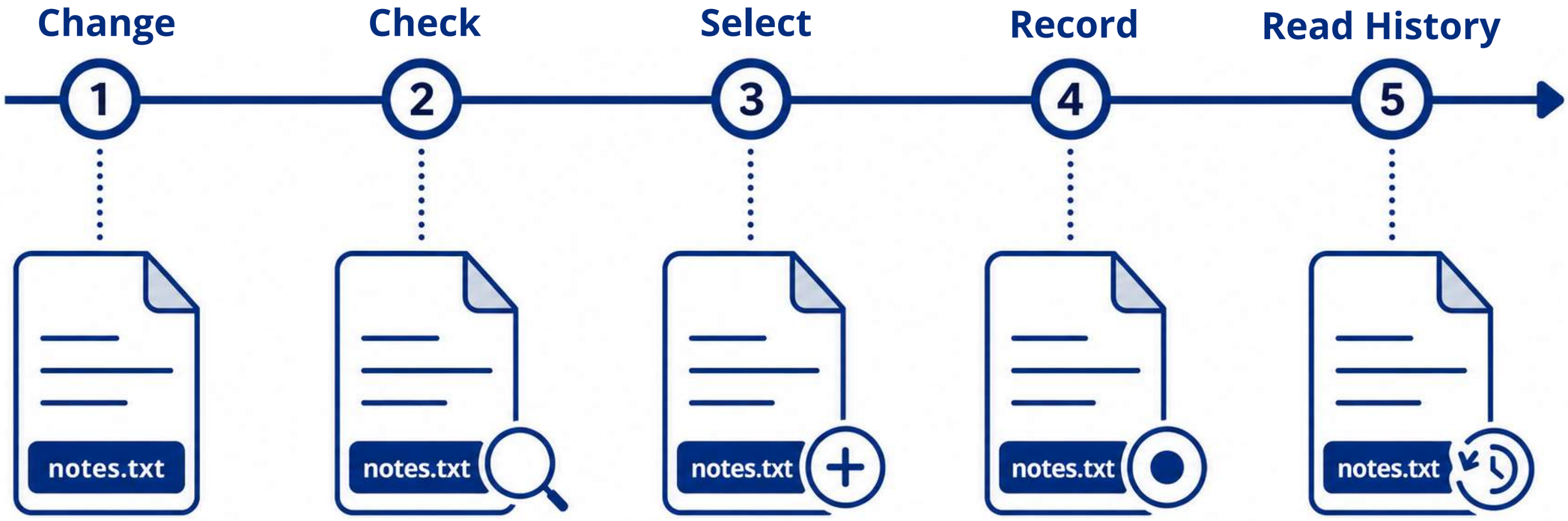
git add

How do you record the change?

git commit -m “<msg>”

How do you read the history?

Can You Rebuild the Workflow



Make a change to the file.

Which command checks Git's status?

git status

How do you select the change?

git add

How do you record the change?

git commit -m “<msg>”

How do you read the history?

git log

Further Reading

Newton School of Technology

- **Git Manual & Command Help (man git, git help commit)**
  [https://git-scm.com/docs**]**
- **Pro Git Book [**https://git-scm.com/book/en/v2**]**
- **Linus Torvalds on Git — Google Tech Talk (2007)**
  [https://www.youtube.com/watch?v=4XpnKHJAok8]
- **Unix Philosophy [**https://en.wikipedia.org/wiki/Unix_philosophy**]**

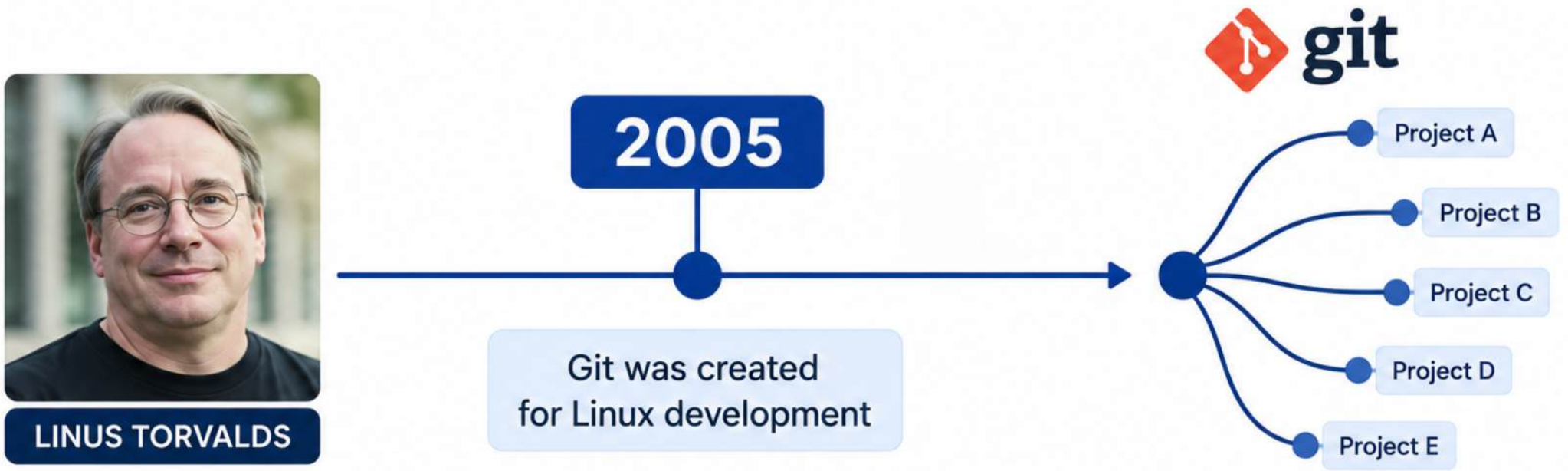
Thanks for watching!

Newton School of Technology

Appendix

Where Git Came From

- Git was created in 2005 by Linus Torvalds — the creator of the Linux kernel.
- He built Git in about two weeks to support Linux development.
- Today, Git is used to build software projects around the world.



Write Clear Commit Messages

Newton School of Technology

- A useful message starts with an **action** and names the **specific change**.

